Indian Institute of Technology Jodhpur

Academic Helpdesk Agent

Design, Implementation and Evaluation of a Multi-Agent Academic Helpdesk

A grounded question-answering and email-drafting system using Google ADK, Ollama-hosted language models, ChromaDB and hybrid RAG

Team member	Roll number
Mohit Meemraauth	B23CS1038
Nisarg Upadhyay	B23CS1075
Divyansh Maheshwari	B23CM1015
Tejas Kalkar	B23CM1051

Project Report
Academic Year 2026–27

Abstract

This report presents the design and evaluation of a local, multi-agent Academic Helpdesk for IIT Jodhpur. The system answers questions about academic regulations, curricula, course offerings and timetables; drafts (but never sends) emails to course instructors; and refuses and logs questions outside its verified knowledge domain. The implementation combines Google Agent Development Kit (ADK), Ollama-hosted 8B language models, ChromaDB, local embeddings and a hybrid dense–BM25 retrieval pipeline. The corpus contains 29 public IIT Jodhpur documents and structured data files for courses, timetable slots, faculty contacts and a student profile.

The evaluation contains 30 questions, three model families, single-agent and multi-agent configurations, and three repeated runs per configuration, producing 630 graded answers. Qwen3:8b is the strongest overall model: the single-agent configuration reaches 80.6% in-scope accuracy, while the multi-agent configuration reaches 79.9% with 94% grounded answers and 100% correct out-of-scope refusals. The multi-agent design improves routing and policy performance, but the thinking-enabled orchestrator increases mean latency from 10.9 s to 30.0 s. The results show that reliable tools, explicit citation requirements, deterministic refusal logging and directory-validated email drafting are as important as model choice.

Keywords: academic helpdesk, multi-agent systems, retrieval-augmented generation, Google ADK, Ollama, ChromaDB, grounded question answering

Contents

1 Introduction **1**

2 Requirements and Scope **1**
2.1 In-scope capabilities 1
2.2 Out-of-scope behavior 1
2.3 Representative questions 2

3 Data Collection and Knowledge Sources **2**
3.1 Document corpus 2
3.2 Structured data and data quality 2

4 System Architecture **2**
4.1 High-level architecture 3
4.2 Agent responsibilities 3

5 Retrieval-Augmented Generation Pipeline **4**
5.1 Indexing 4
5.2 Hybrid retrieval 4
5.3 Initial retrieval check 5

6 Implementation **5**
6.1 Local execution 5
6.2 Grounding and citation policy 5
6.3 Email safety 6

7 Evaluation Methodology **6**
7.1 Question set 6
7.2 Configurations and repetitions 6
7.3 Metrics 6

8 Results **6**
8.1 Overall comparison 7
8.2 Accuracy by question category 8
8.3 Deterministic checks 8

9 Analysis of Findings **8**
9.1 Qwen3 is the strongest practical choice 8
9.2 Specialisation is not automatically beneficial 9
9.3 Thinking in the orchestrator fixes silent routing failures 9
9.4 Out-of-scope refusal and logging are strong 9
9.5 Granite is unsuitable for this tool-calling design 9

10 Failure Modes and Limitations **9**

11 Recommendations and Future Work **10**

12 Reproducibility **10**

13 Conclusion **10**

A Evaluation Question Categories **11**

B References **11**

1 Introduction

Students regularly need small but consequential academic facts: the attendance rule for an examination, the meeting time of a course, the number of credits required for graduation, or the correct authority to whom a leave application should be addressed. These facts are distributed across regulations, circulars, curriculum documents, calendars, timetable sheets and faculty directories. A general-purpose chatbot can answer fluently while silently mixing old and new rules, inventing a timetable, or hallucinating an instructor's email address.

The objective of this project is therefore not simply to build a conversational interface. It is to build a bounded academic information system that:

1. answers questions only when evidence is available in an institutional corpus or an approved structured data source;
2. cites the source used for the answer;
3. routes questions to specialists for policy, schedule and email tasks;
4. refuses and logs out-of-scope questions instead of guessing;
5. drafts emails for student review without sending them; and
6. remains executable on a laptop with approximately 16 GB RAM using local models.

The rest of this report describes the corpus, architecture, retrieval and agent design, evaluation methodology, quantitative results, observed failure modes and recommendations for a production-quality deployment.

2 Requirements and Scope

2.1 In-scope capabilities

The helpdesk supports four functional categories:

- **Policy and curriculum:** academic regulations, attendance, examination rules, degree credits, add/drop dates and academic calendars.
- **Schedule:** course lookup, slot and room lookup, student timetable queries and counting courses by department, level or semester.
- **Email assistance:** instructor contact lookup and drafting of leave or make-up examination emails.
- **Safety and escalation:** refusal of unsupported questions and persistent logging for staff follow-up.

2.2 Out-of-scope behavior

Questions about dining options, travel directions, campus facilities, placement statistics or other topics not represented in the corpus must receive a refusal rather than an inferred answer. A refusal is treated as a successful behavior when the question is genuinely outside the verified domain. It must also be logged in `logs/unanswered.jsonl`.

2.3 Representative questions

Table 1: Representative questions covered by the system.

Category	Example
Policy	What is the attendance requirement to sit in the Major exam?
Schedule	What is the slot for the course Autonomous Systems?
Counting	How many 400-level electives have been floated by the Humanities department this semester?
Degree	How many Open Elective credits are required for a B.Tech. degree?
Email	Write an email to the instructor of CSL7570 that I need leave for two days.
Out of scope	What are the available dining options on campus?

3 Data Collection and Knowledge Sources

3.1 Document corpus

The project collected 29 public IIT Jodhpur sources: 24 PDFs and five web pages. The sources cover academic regulations, attendance and examination amendments, academic calendars, holidays, fees, scholarships, medical forms, B.Tech./M.Tech. curricula, common courses, non-graded courses and humanities courses. The manifest records document title, category, source URL, fetch date, page count and a content hash.

Table 2: Knowledge sources used by the helpdesk.

Source type	Count	Examples and role
PDF documents	24	Regulations, curricula, circulars, calendars, fees, scholarships and forms.
Web pages	5	Academic programmes, registration, engineering/science information and contact pages.
Structured CSV/JSON	4	Course offerings, timetable, faculty directory and student profile.
Total public documents	**29**	Indexed for retrieval and citation.

3.2 Structured data and data quality

Structured lookup tools complement document retrieval. The course catalogue describes Semester I of AY 2026–27, the official slot system stores standard meeting times, the faculty directory supplies instructor contacts, and the student profile enables questions such as “What classes do I have on Friday?” Course titles, levels, credits and several faculty facts are sourced from IIT Jodhpur material. Course codes not published by the source documents, course-to-instructor assignments and some slot/room mappings are explicitly marked illustrative or synthetic in the repository documentation. This distinction prevents synthetic test data from being mistaken for official institutional policy.

4 System Architecture

4.1 High-level architecture

The final system has two modes. The baseline is a single ADK `LlmAgent` with all tools. The proposed multi-agent mode uses an orchestrator with no domain tools; it transfers each request to a policy, schedule or email specialist. The same retrieval and structured tools are used in both modes.

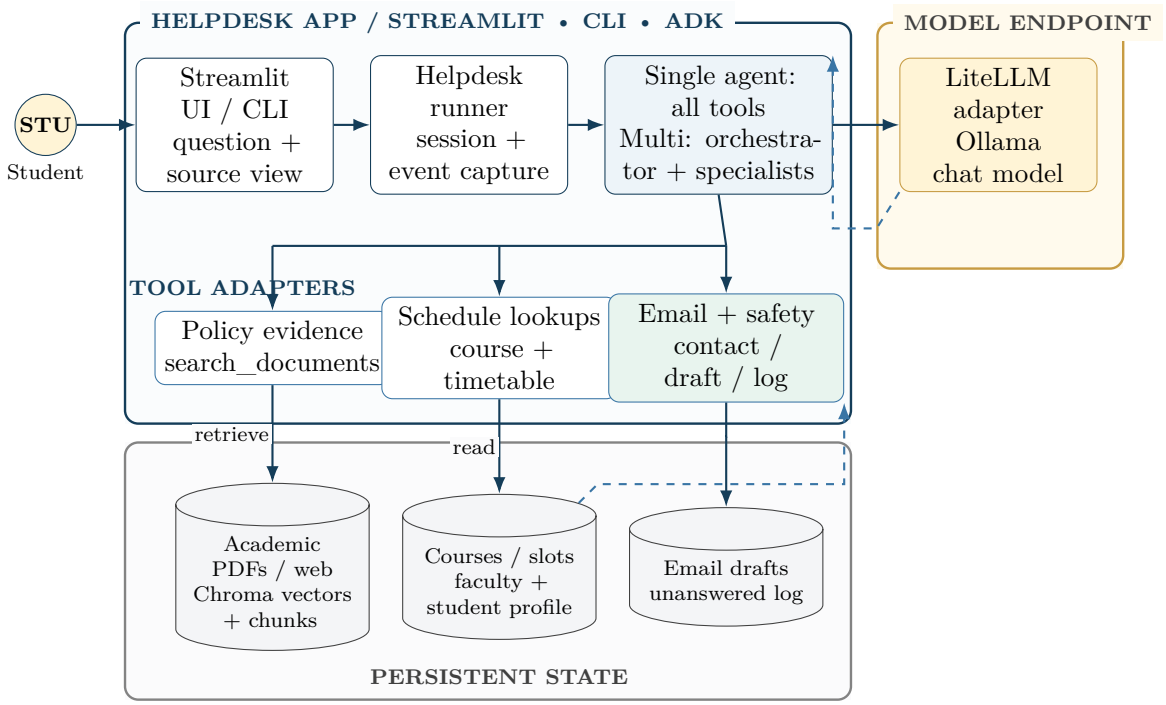


Figure 1: Runtime architecture and trust boundaries. The application contains the interface, runner, agent mode and tool adapters. Ollama serves model inference separately; the retriever, structured catalogues and generated files form distinct data paths.

4.2 Agent responsibilities

Table 3: Agents and allowed responsibilities.

Agent	Responsibility and tools
Helpdesk orchestrator	Understands the request, transfers to one specialist, and refuses/logs unsupported topics.
Policy agent	Searches regulations, curricula, circulars and calendars using `search_documents`.
Schedule agent	Uses `find_course`, `list_courses`, `get_slot_timetable` and `get_my_schedule`.
Email agent	Uses `get_instructor_contact` and `draft_email`; it never sends messages.
Safety callback	An `after_agent_callback` logs refusals that the model fails to log explicitly.

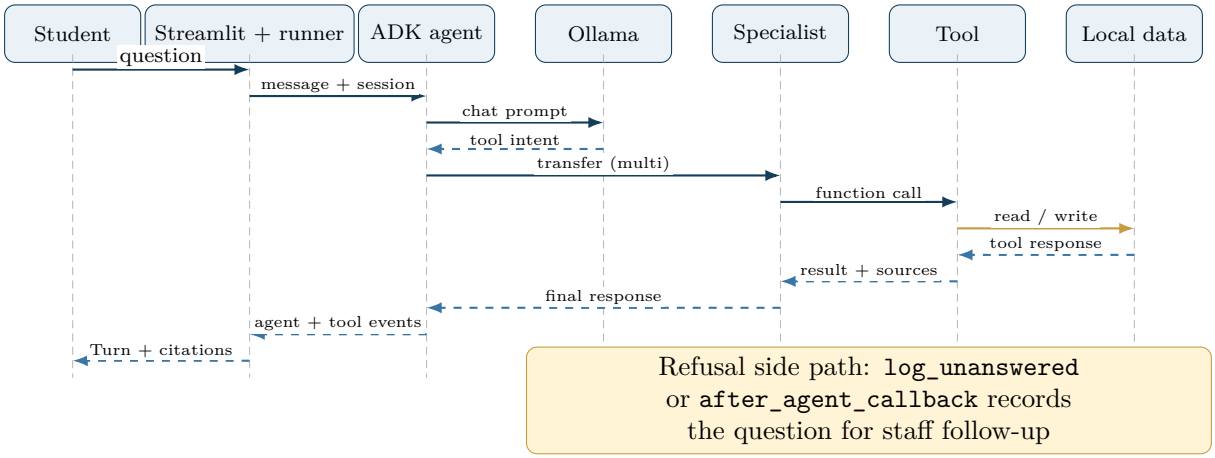


Figure 2: Turn sequence through the runtime. Multi-agent mode adds the specialist transfer; single-agent mode calls the tools directly. In both modes, the runner captures agent and tool events before returning an answer, citations and any saved draft.

5 Retrieval-Augmented Generation Pipeline

5.1 Indexing

PDF and web content is normalized, with scanned PDFs passed through OCR when necessary. Text is split into approximately 350-word, section-aware chunks. A one-paragraph overlap reduces boundary loss, while page ranges are retained for citations. Repeated headers and footers are removed before embedding.

5.2 Hybrid retrieval

Each query is processed by two complementary retrievers:

1. dense retrieval using the local `nomic-embed-text` model and ChromaDB; and
2. lexical retrieval using BM25, which is particularly useful for exact course codes, clause numbers and dates.

The ranked lists are fused with reciprocal rank fusion. A minimum dense similarity threshold is used as an early signal that a question may not be answerable from the corpus. The model prompt still has to verify that the returned passages answer the question.

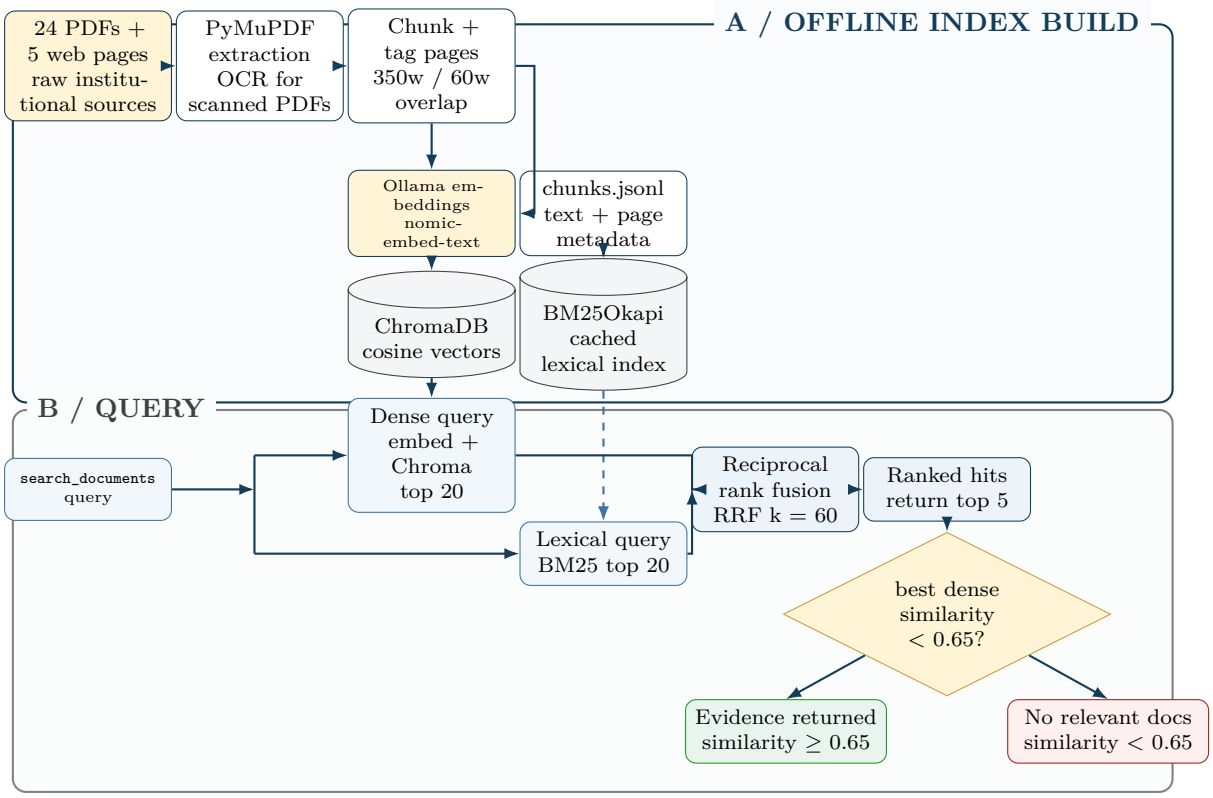


Figure 3: Hybrid retrieval architecture. Index construction preserves page metadata and builds parallel vector and BM25 indexes. At query time, both ranked lists are fused; the dense-similarity gate returns evidence or triggers the no-relevant-documents refusal path.

5.3 Initial retrieval check

Before agent evaluation, the retrieval pipeline was checked on 25 retrieval questions. It achieved 100% hit@5 and 0.96 mean reciprocal rank. This indicates that the later failures are primarily related to routing, tool use, answer composition, model behavior or source reconciliation rather than a complete inability to retrieve the relevant document.

6 Implementation

6.1 Local execution

The project is packaged with `uv` and supports Python 3.11–3.13. Ollama hosts the selected 8B chat models and local embedding model. A remote GPU server may be reached through an SSH tunnel, but the Ollama endpoint must not be exposed publicly because the service has no authentication. The intended laptop deployment uses approximately 16 GB RAM for one 8B model.

6.2 Grounding and citation policy

The agent prompt requires a tool call before an academic answer and requires citations in square brackets. When sources conflict, the newest circular or amendment is intended to supersede older regulations. The agent should not claim a draft was saved unless `draft_email` actually ran; the UI displays saved drafts verbatim from the tool output.

6.3 Email safety

The email tool accepts only instructor addresses present in the faculty directory. It writes a draft file and never sends an email. The generated draft contains a subject, recipient, request details and the student's signature. This design limits the impact of a mistaken model completion and leaves the final communication under student control.

7 Evaluation Methodology

7.1 Question set

The evaluation set contains 30 questions:

- 12 policy questions;
- 8 schedule questions;
- 4 instructor-contact or email-drafting questions; and
- 6 out-of-scope questions, including near-misses such as hostel-mess attendance and the current Director.

Each question includes a reference answer, gold source identifiers, expected specialist and expected tools.

7.2 Configurations and repetitions

The experiment compares `qwen3:8b`, `llama3.1:8b` and `granite3.3:8b` in single-agent and multi-agent modes. Qwen3 also has a `multi_nothink` ablation in which the orchestrator's thinking mode is disabled. Every configuration runs all 30 questions three times with fresh sessions, resulting in:

$$30 \times (3 \text{ models} \times 2 \text{ modes} + 1 \text{ ablation}) \times 3 = 630 \text{ graded answers.}$$

7.3 Metrics

The answer judge evaluates correctness, groundedness, citation quality and refusal. Deterministic checks evaluate refusal, unanswered logging, routing, tool selection, citation-to-returned-source matching, gold-source retrieval and latency. For in-scope correctness, a correct answer scores 1, a partial answer 0.5 and an incorrect answer 0. Out-of-scope refusal is successful only when the system refuses and logs the question.

8 Results

8.1 Overall comparison

Table 4: Main evaluation results. Percentages are means over three runs.

Model	Mode	Acc.	Ground.	Cite.	Refuse	Log	Route	Tools	Consist.	Latency
Qwen3:8b	Single	80.6	97.0	79.6	100	100	–	88.0	80.0	10.9/19.5
Qwen3:8b	Multi	79.9	94.0	90.0	100	100	96.7	88.9	86.7	30.0/53.5
Qwen3:8b	Multi, no think	72.9	88.9	89.5	100	100	90.0	83.3	93.3	13.0/23.0
Llama3.1:8b	Single	71.5	70.0	76.3	100	100	–	90.3	80.0	14.3/27.0
Llama3.1:8b	Multi	60.4	75.0	71.4	100	100	94.4	81.9	56.7	12.2/22.8
Granite3.3:8b	Single	6.9	4.2	3.4	55.6	55.6	–	8.3	26.7	10.8/25.2
Granite3.3:8b	Multi	0.0	98.6†	71.7†	55.6	55.6	20.0	0.0	83.3†	2.4/4.9

Acc. = in-scope accuracy; *Ground.* = groundedness; *Cite.* = citation correctness; *Refuse* = correct out-of-scope refusal; *Log* = out-of-scope logging; *Consist.* = questions judged equivalent across repetitions; latency is mean/p90 in seconds. †Granite multi mainly produced pseudo-tool-call text rather than useful answers, making these judge metrics misleading.

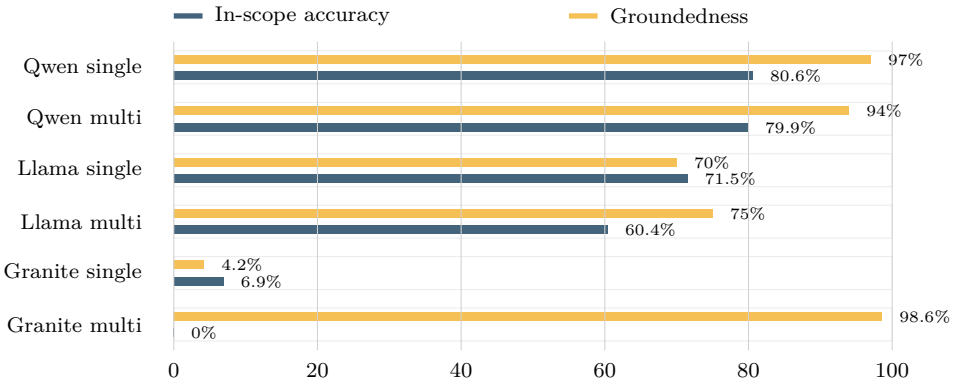


Figure 4: In-scope accuracy and groundedness. Granite multi groundedness is high only because pseudo-tool text contains few factual claims; it does not indicate useful answers.

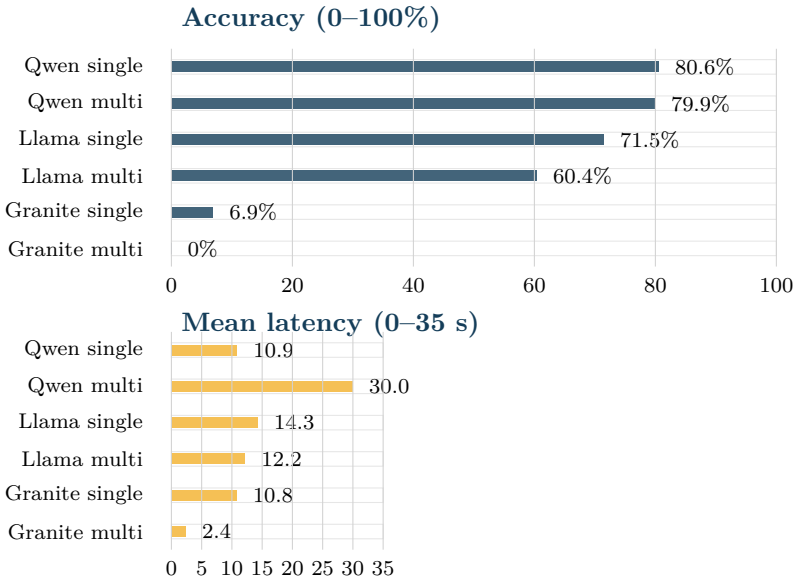


Figure 5: Accuracy and mean response latency on separate scales.

8.2 Accuracy by question category

Table 5: Accuracy by category (percent).

Model	Mode	Policy	Schedule	Email	Out of scope
Qwen3:8b	Single	67	100	83	100
Qwen3:8b	Multi	81	98	42	100
Qwen3:8b	Multi, no think	68	98	38	100
Llama3.1:8b	Single	68	98	29	100
Llama3.1:8b	Multi	62	83	8	100
Granite3.3:8b	Single	6	8	8	94
Granite3.3:8b	Multi	0	0	0	89

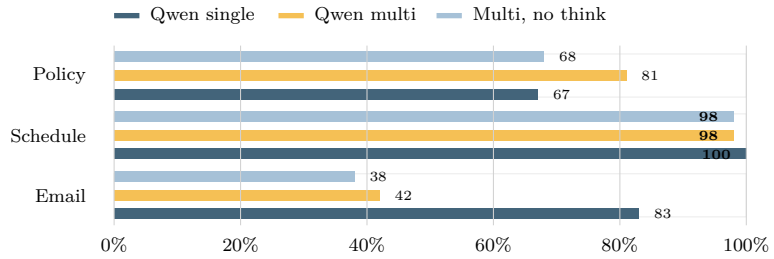


Figure 6: Qwen3 category-wise comparison. Specialisation improves policy accuracy but reduces email accuracy in this implementation.

8.3 Deterministic checks

Table 6: Deterministic tool and retrieval checks (percent, except counts).

Model	Mode	Cites returned source	Gold source	Empty	Errors
Qwen3:8b	Single	85	89	0	0
Qwen3:8b	Multi	100	90	0	0
Qwen3:8b	Multi, no think	95	83	6	0
Llama3.1:8b	Single	98	94	0	0
Llama3.1:8b	Multi	95	86	0	0
Granite3.3:8b	Single	2	8	0	0
Granite3.3:8b	Multi	0	0	0	0

9 Analysis of Findings

9.1 Qwen3 is the strongest practical choice

Qwen3:8b is the only model that combines high answer quality, strong grounding and perfect refusal behavior in both modes. The single agent has the best measured accuracy and substantially lower latency. The multi-agent version has slightly lower accuracy but improves citation correctness from 80% to 90%, routing accuracy to 96.7% and run consistency to 86.7%. Thus, the multi-agent design is justified when modularity, observability and specialist separation are priorities, while the single agent is the better latency-sensitive deployment.

9.2 Specialisation is not automatically beneficial

The multi-agent version improves Qwen3's policy performance from 67% to 81%, likely because the policy specialist has a narrower prompt and a focused tool set. The same decomposition hurts Llama3.1:8b, whose accuracy falls from 71.5% to 60.4%. An additional transfer creates another opportunity for malformed arguments, premature refusal or failure to continue after routing.

9.3 Thinking in the orchestrator fixes silent routing failures

With Qwen3's orchestrator thinking disabled, two ambiguous questions repeatedly returned empty messages. Enabling thinking only for the orchestrator reduced empty answers from six to zero, increased routing accuracy from 90% to 96.7% and increased in-scope accuracy from 72.9% to 79.9%. The cost was mean latency increasing from 13.0 s to 30.0 s and p90 latency increasing from 23.0 s to 53.5 s.

9.4 Out-of-scope refusal and logging are strong

Qwen3 and Llama3.1 refused 100% of out-of-scope questions and logged them 100% of the time. The callback was essential: Qwen3 frequently produced the refusal but did not call `log_unanswered` itself, so the callback provided the safety guarantee. False refusals remain a concern, ranging from 3% to 12% for Qwen3 and Llama3.1.

9.5 Granite is unsuitable for this tool-calling design

Granite3.3:8b did not reliably emit structured tool calls through the Ollama configuration. In multi-agent mode it often printed a pseudo transfer instruction as the answer. Its apparent 98.6% groundedness is therefore not evidence of useful performance: the output often contained no factual claim at all. A one-tool probe should be mandatory before replacing the selected model.

10 Failure Modes and Limitations

1. **Email ambiguity:** Questions asking both for procedural policy and an email can be routed to the wrong specialist. Qwen3 multi-agent email accuracy is only 42%, compared with 83% for the single agent.
2. **Superseded regulations:** The July 2024 attendance and examination circulars supersede older rules. Models sometimes combine the old and new policy instead of explicitly preferring the amendment.
3. **Synthetic structured fields:** Some course codes, room assignments and course-to-instructor mappings are illustrative. The report and UI must label these fields clearly.
4. **Small evaluation set:** The experiment contains only 30 questions and three repeated runs. Differences of a few percentage points may reflect sampling noise.
5. **Latency:** A thinking orchestrator adds nearly three times the mean latency of Qwen3 single-agent mode. This may be unacceptable for interactive use without streaming or caching.
6. **External judge dependency:** The evaluation judge uses Claude Sonnet 5.5. The judge itself can introduce subjectivity, especially for partial answers and citation quality.

11 Recommendations and Future Work

1. Deploy Qwen3:8b single-agent mode where latency is the primary objective; deploy multi-agent mode where specialist modularity and routing telemetry are more important.
2. Require the exact course name when an email query does not contain a course code, and make `not_found` responses suggest retrying with the title.
3. Add document dates and explicit "superseded by" metadata to retrieval chunks, with a ranking boost for newer circulars.
4. Reject any user-facing statement that a draft was saved unless the runner has a corresponding successful `draft_email` event.
5. Expand the question set with paraphrases, multilingual queries, adversarial near-misses and more conflicts between old and new regulations.
6. Add retrieval confidence, source freshness and tool traces to the evaluation dashboard.
7. Use a structured output schema for every specialist and validate the schema before returning an answer.
8. Add a human review workflow for unanswered questions and resolved staff responses, allowing the corpus to improve over time.

12 Reproducibility

The repository contains the source code, data manifest, structured data, question set, raw evaluation runs, judge verdicts and scoring scripts. A fresh setup is:

```
uv sync --python 3.12
cp .env.example .env
uv run python scripts/fetch_docs.py
uv run python scripts/ocr_scans.py
uv run pytest -q
uv run python -m helpdesk.ingest
uv run python scripts/check_retrieval.py
uv run python eval/run_eval.py
uv run python eval/judge.py
uv run python eval/score.py
```

The interactive modes are:

```
uv run python -m helpdesk.cli "What is the slot for the course Autonomous Systems?"
uv run python -m helpdesk.cli --mode multi
uv run streamlit run app/streamlit_app.py
```

13 Conclusion

This project demonstrates a practical pattern for a bounded institutional assistant. The combination of retrieval, structured tools, specialist agents, citations, refusal logging and directory validation is more reliable than unconstrained generation. The evaluation confirms that Qwen3:8b is the most suitable tested model for the current tool-calling stack. The single-agent configuration is the best baseline for accuracy and latency, while the multi-agent configuration provides better modularity, routing visibility and citation behavior at a substantial latency cost.

The most important engineering lesson is that deterministic safeguards must remain outside the language model: callbacks must log refusals, tools must validate recipients, and the runner must verify that claimed actions actually occurred.

A Evaluation Question Categories

Table 7: Question identifiers and examples from the evaluation set.

ID	Question
P01	What is the attendance requirement to sit in the Major exam?
P02	How many Open Elective credits do I need to earn in order to get a B.Tech. Degree?
P03	I missed my Major exam because I was ill. How do I apply for a make-up exam?
P04	When are the Major examinations for Semester I of 2026–27?
P05	How long is the Major examination and what is its minimum weightage?
P06	What happens if I am absent from all academic activities for four weeks in a semester?
P07	What is the last date to add or drop courses in Semester I 2026–27?
P08	To whom should a B.Tech. student address an application for leave of absence?
P09	How many total credits are required for a regular B.Tech. degree?
P10	How many credits must a B.Tech. student earn by the end of the second semester?
P11	When is the Minor examination in Semester I 2026–27?
P12	What is the maximum duration allowed to complete a B.Tech. programme?
S01	What is the slot for the course Autonomous Systems?
S02	How many 400-level electives have been floated by the Humanities department?
S03	What classes do I have on Friday?
S04	Which room is Computational Game Theory held in?
S05	On which days and at what time does slot F meet?
S06	How many 700-level electives is the Computer Science department offering?
S07	How many credits is Environmental Science and what is its L-T-P structure?
S08	When is CSL7570 scheduled?
E01–E04	Instructor contact and email-drafting requests, including leave and make-up exam emails.
O01–O06	Out-of-scope questions about dining, campus travel, facilities, current officials and placement statistics.

B References

1. IIT Jodhpur academic regulations, curricula, examination circulars and academic calendars collected in `data/raw/`.

2. Academic Helpdesk Agent repository, https://github.com/Mohi1038/academic-helpdesk-agent.
3. Google Agent Development Kit documentation, https://google.github.io/adk-docs/.
4. ChromaDB documentation, https://docs.trychroma.com/.
5. Ollama documentation, https://ollama.com/.